

An Empirical Analysis of Technical Debt in Open-Source Machine Learning Repositories

Student: Vidhumol Pandippilly Antony

Supervisor: Dr.sc.ing. Boriss Misnevs

Agenda

1. Introduction
2. Problem and Motivation
3. Research Aim and Research Questions
4. Methodology
5. Key Finding
6. Limitations Future Work
7. Conclusions
8. Acknowledgements

Introduction

- **What is Technical Debt?** Coined by Ward Cunningham (1992) - the future maintenance cost of choosing an expedient solution over an optimal one. Like financial debt, deferred fixes accrue "interest."
- **Self-Admitted Technical Debt (SATD)** Developers explicitly admit shortcuts in source code comments: # TODO: fix this properly · # HACK: temporary workaround · # FIXME: breaks on edge case First formalised by Potdar & Shihab (2014) -found in up to **31% of files** in large open-source projects.
- **Why ML systems are different** (*Sculley et al., 2015 - NeurIPS*) ML systems accumulate debt faster and in entirely different forms: data dependency debt, pipeline jungle debt, configuration debt, boundary erosion - none of which have analogues in traditional software.

Introduction

- **Research Aim:** To conduct a multi-modal empirical analysis of Self-Admitted Technical Debt (SATD) to understand its volume, composition, and severity across five distinct machine learning sub-categories.
- **Object of Research:** The phenomenon of technical debt (specifically SATD) within the lifecycle of open-source machine learning software development.
- **Subject of Research:** The comparative patterns, distribution, and semantic characteristics of SATD in 526 repositories belonging to Deep Learning, NLP, Computer Vision, MLOps, and AutoML/RL domains.

Introduction

- **Research Tasks:**
 - Collect a diverse corpus of 526 ML and non-ML repositories.
 - Extract SATD instances from Python and non-Python artifacts using an expanded set of keywords.
 - Develop and apply an LLM-based semantic classifier to determine debt categories and severity.
 - Utilize unsupervised clustering to discover latent behavioral debt patterns.
 - Statistically validate differences in debt volume and composition across ML sub-categories.

PROBLEM AND MOTIVATION

Five Critical Blind Spots in Prior SATD Studies

Table 1: Research Gaps Addressed by This Study

Gap	Prior State	This Study
Sub-category analysis	No comparison across DL, NLP, CV, MLOps, and AutoML	First statistically validated sub-category comparison
Python-only scanning	YAML, notebooks, and Dockerfiles ignored	Four non-Python artifact types scanned
No severity grading	Only presence/absence detected	LLM assigns severity scores (1–5) per comment
Manual taxonomies	Card-sorting with limited scale	Automated K-means unsupervised discovery
Weak classifiers	NLP detector with a 44% false positive rate	Llama 3.1 achieves a 0% false positive rate

Motivation from prior work:

- Bhatia et al. (2025) — ML projects have **2× SATD density** vs non-ML; **140× faster** debt introduction
- Pepe et al. (2024) — 41 DL-specific SATD types across 54 repositories
- Selvanayagam et al. (2026) — LLM repos accumulate SATD at similar rate to ML (3.95% vs 4.10%)

Research Aim & Research Questions

Aim: First multi-modal, statistically validated empirical study of SATD across five ML sub-categories — 526 repositories, 12-notebook pipeline.

Table 2: Five Research Questions

RQ	Question	Answer
RQ1	Are there statistically significant SATD volume differences across ML sub-categories?	Yes — $H = 25.26$, $p = 0.000045$
RQ2	Are there new debt types in MLOps and LLM repositories?	Yes — 6 novel categories and 2 unnamed clusters were identified
RQ3	Does Python-only analysis underestimate total ML debt?	Yes — a 4.1% underestimation was confirmed
RQ4	Does a composite model outperform single-signal debt prediction?	Yes — Random Forest achieved 90.4% accuracy versus 73.1% for Logistic Regression
RQ5	Do ML-specific debt patterns differ from those in traditional software?	Yes — $\chi^2 = 1204.66$, $p \approx 0$, indicating domain-specific patterns

Methodology

Table 3 : 12-Notebook Pipeline Architecture

Phase	Notebooks	Purpose	Output
Baseline	NB01–NB05	Repository collection, SATD extraction, static analysis, commit history, ML classifier	master_dataset.csv
Innovation	NB06–NB09	LLM classifier, K-means clustering, multi-file scan, statistical validation	llm_classified_results.csv, cluster_results.csv
Extension	NB10–NB12	Scale to 526 repositories, replicate pipeline, combined validation and SDLC analysis	nb12_full_validation.png

METHODOLOGY

Table 4: Repository Dataset — 526 Open-Source Repositories

ML Sub-Category	Count	Key Examples
Deep Learning	112	tensorflow, pytorch, vllm, llama.cpp
NLP	94	langchain, llama_index, autogen
Computer Vision	78	yolov5, segment-anything, ComfyUI
MLOps	87	mlflow, wandb, dagster, airflow
AutoML / RL	75	optuna, gymnasium, LightGBM
Non-ML Control	80	flask, django, pytest, ruff
Total	526	

Methodology

SATD Extraction — Table 5: 10-Keyword Taxonomy (7,753 total records)

Keywords	Debt Type	Novel?
TODO, FIXME, HACK, XXX	Requirement Debt / Defect Debt / Design Debt	Standard
BUG, WORKAROUND, TEMP, DEBT, SMELL	Defect Debt / Design Debt / Code Debt	Extended
NOQA	Test Debt	Novel — first identified in SATD literature

LLM Classifier: Llama 3.1 via Groq API → 2,803 comments → 19-category taxonomy → severity 1–5 **Statistical tests:** Kruskal-Wallis · Chi-Square · Dunn post-hoc (Bonferroni) · Spearman · K-means (TF-IDF, k=8)

Key Finding: RQ1 — SATD Volume Across Sub-Categories

RQ1 Answer: YES — statistically significant at $p=0.000045$

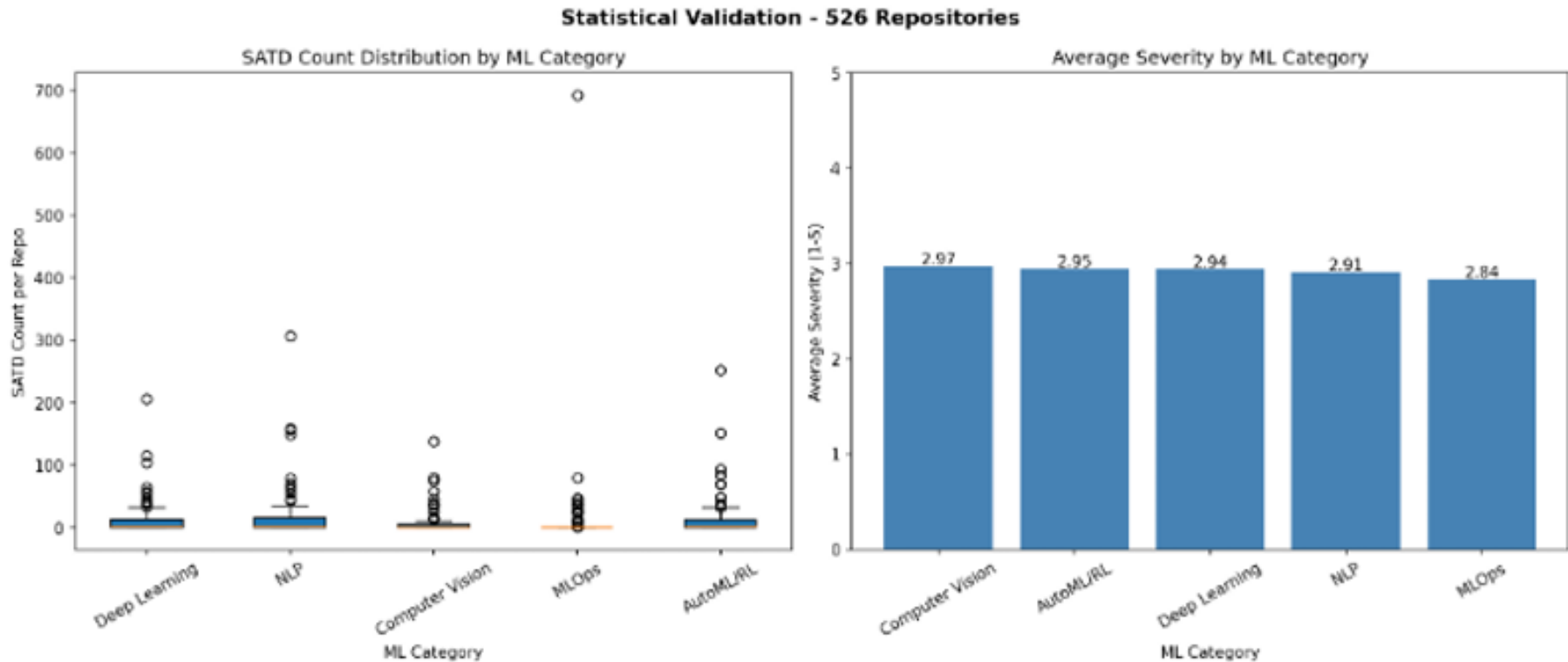


Figure 1: SATD Count Distribution and Average Severity by ML Category — 526 Repositories

Key Finding: RQ1 — SATD Volume Across Sub-Categories

Table 6: Kruskal-Wallis Test Result

Test	Statistic	p-value	Conclusion
Kruskal-Wallis H-test	H = 25.26	p = 0.000045	SATD volume differs significantly across ML sub-categories

Table 7: Dunn Post-Hoc Test — Significant Pairs

Pair	p-value	Significant?
MLOps vs NLP	0.0003	Most significant
MLOps vs Deep Learning	0.0015	✓
MLOps vs AutoML/RL	0.0048	✓
All other pairs	> 0.05	✗

Key insight: MLOps is the statistical outlier — bimodal distribution (median=0.0, mean=12.61, std=74.96). Many zero-SATD repos alongside high-count orchestration tools (Airflow, Prefect, DVC).

Key Finding: RQ2 — New Debt Types

RQ2 Answer: 6 novel debt categories + 2 previously unnamed behavioural clusters

Table 8: Six Novel ML-Specific Debt Categories (LLM-Confirmed, NB06 + NB11)

New Debt Category	Instances	Domain	What it Captures
Experiment Tracking Debt	5	MLOps	Deferred metric logging and hardcoded experiment names in MLflow, W&B, and ClearML
Prompt Template Debt	7	NLP	Hardcoded prompt strings and deferred prompt engineering in LangChain and LlamaIndex
Data Versioning Debt	Confirmed	MLOps	Missing dataset lineage and hardcoded paths in DVC and Feast
Monitoring & Observability Debt	Confirmed	MLOps	Missing drift-detection thresholds in Evidently AI and WhyLogs
MLOps Pipeline Debt	1	MLOps	Deferred pipeline orchestration fixes
Token Budget Debt	Confirmed	NLP / LLM	Unmanaged context window costs

Key Finding: RQ2 — New Debt Types

Table 9: Two Previously Unnamed Behavioural Clusters (K-means, NB07)

Cluster Name	Size	Domain	Discovery Method
Cluster 2: Path Management Debt	276 instances	MLOps (near-exclusive)	Unsupervised K-means clustering
Cluster 5: Import Suppression Debt	187 instances	NLP-dominant	Enabled by the novel NOQA keyword

Key Finding: RQ3 — Python-Only Underestimation

RQ3 Answer: YES — 4.1% underestimation confirmed across all 526 repositories

Table 10: Python vs Non-Python SATD Comparison — 526 Repositories (NB08 + NB11)

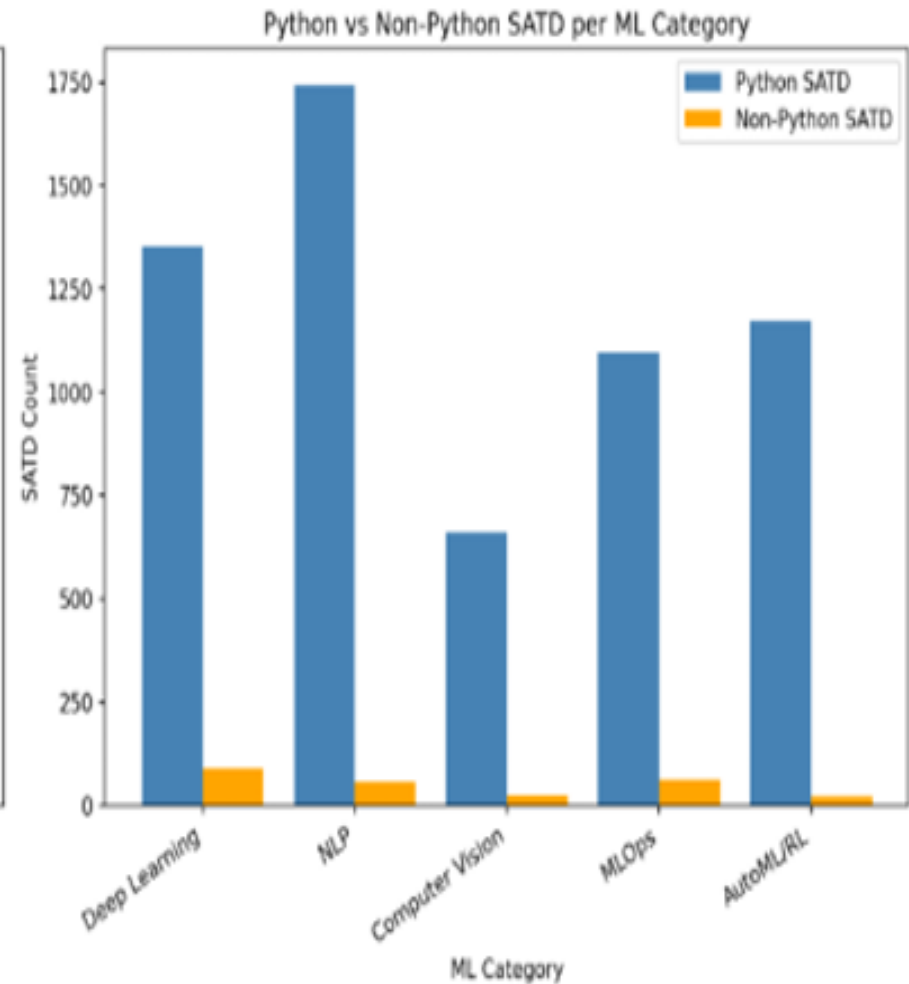
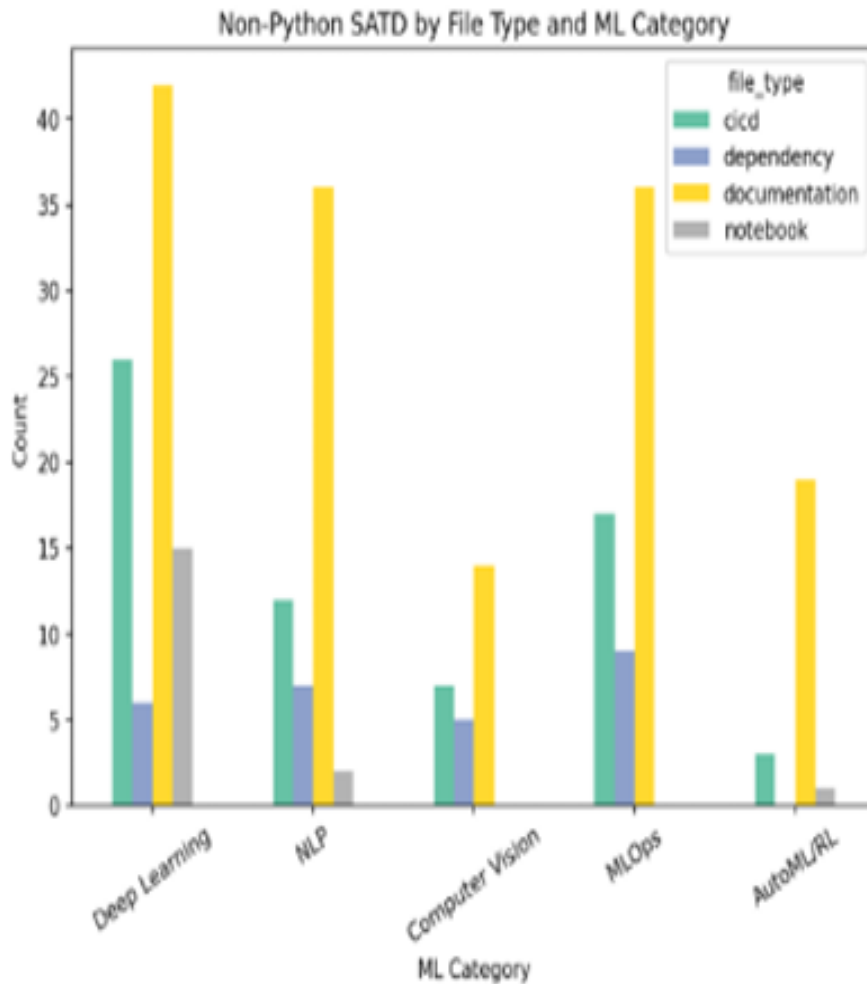
ML Category	Python SATD	Non-Python SATD	Total SATD	Underestimate %
Deep Learning	1,353	89	1,442	6.2% ← Largest
MLOps	1,097	62	1,159	5.3%
Computer Vision	661	26	687	3.8%
NLP	1,743	57	1,800	3.2%
AutoML / RL	1,173	23	1,196	1.9%
Overall	6,027	257	6,284	4.1%

Non-Python breakdown: Documentation files 162 (55%) · CI/CD files 80 (27%) · Dependency files 35 (12%) · Jupyter notebooks 18 (6%)

Implication: Bhatia et al.'s 2× SATD density finding is based on Python only — the true ratio is higher. P7 Documentation and P5 CI/CD debt are **entirely invisible** without multi-artifact scanning.

Key Finding: RQ3 — Python-Only Underestimation

Multi-File-Type SATD Analysis — 526 Repositories



Key Finding: RQ3 — Python-Only Underestimation

RQ4 Answer: Composite Random Forest (90.4%) significantly outperforms single-signal Logistic Regression (73.1%)

Table 11: Baseline Classifier Performance (NB05, 3-fold cross-validation)

Classifier	Accuracy	Key Features
Random Forest	90.4%	SATD count, Flake8 violations, complexity metrics, and issue resolution days
Logistic Regression	73.1%	SATD count and complexity metrics

Table 12: Spearman Correlation Results — SATD Predictors (NB09 + NB12)

Variable Pair	Spearman ρ	p-value	Interpretation
SATD vs Total Commits	0.369	< 0.0001	Strongest predictor — repository churn drives technical debt
SATD vs Repo Stars	0.115	0.008	Popular repositories accumulate slightly more debt
SATD vs Repo Forks	0.101	0.020	Wider adoption is associated with slightly more debt
SATD vs Repo Age	0.054	0.213	✗ Repository age alone does not predict debt
SATD vs Average Severity	-0.400	0.505	✗ High debt volume does not necessarily imply high debt severity

Key Finding: RQ5 — ML-Specific vs Traditional Software Patterns

RQ5 Answer: YES — domain-specific debt signatures confirmed, $\chi^2=1204.66$, $p\approx 0$

Table 13: Chi-Square Test Result — Debt Type vs ML Category

Test	Statistic	df	p-value	Conclusion
Chi-Square Test	1204.66	12	$p = 1.72 \times 10^{-250}$	Debt type distribution is significantly domain-specific

Table 14: Debt Category Distribution by Domain — 2,803 LLM-Classified Comments

Debt Category	Total	%	Primary Domain
Code Debt	1,989	70.9%	All categories
Test Debt	439	15.7%	All categories
Inadequate Testing	95	3.4%	MLOps (47/95)
Defect Debt	93	3.3%	All categories
Configuration Debt	35	1.2%	Deep Learning, MLOps
Prompt Template Debt	7	0.2%	NLP only

Key Finding: RQ5 — ML-Specific vs Traditional Software Patterns

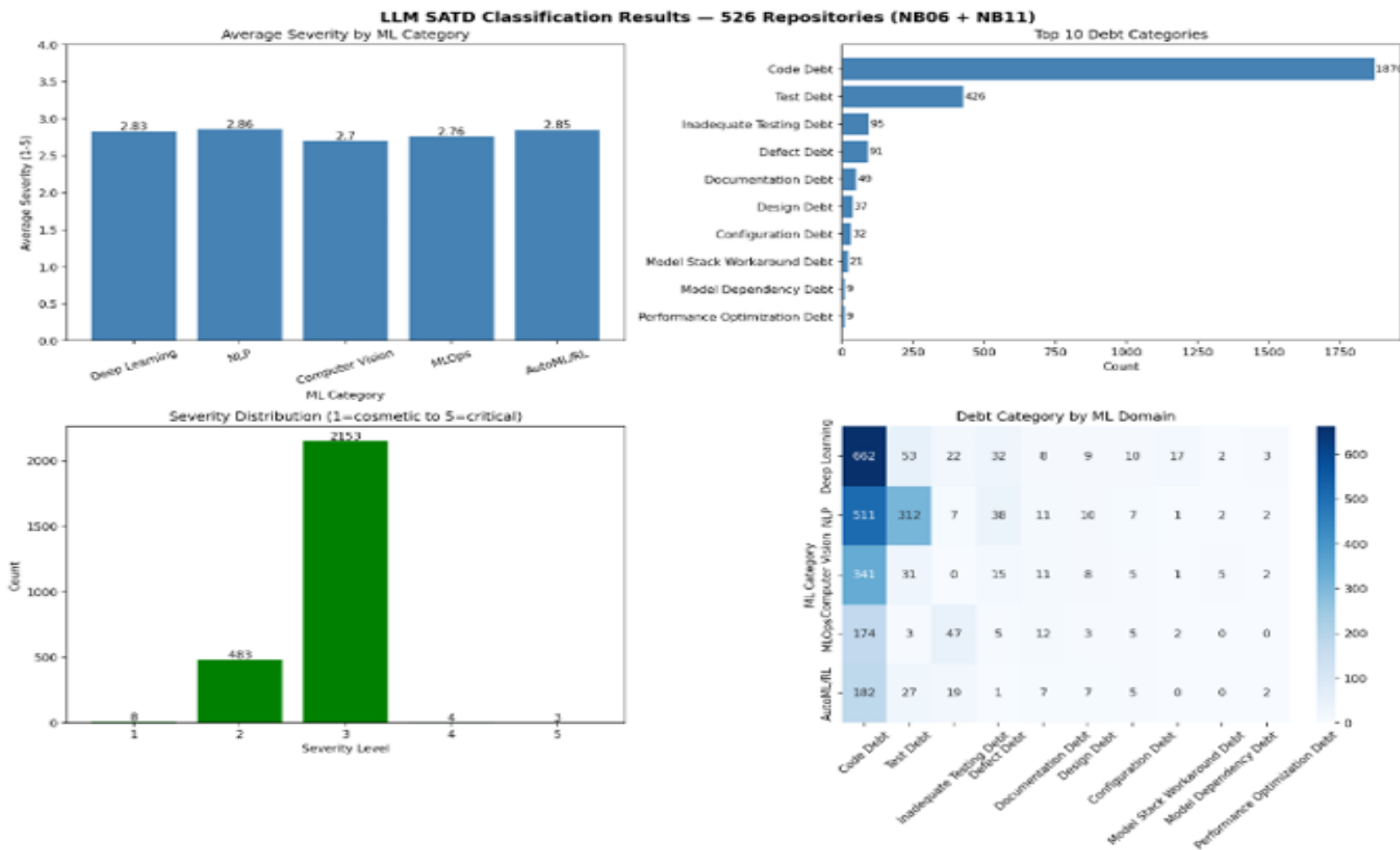


Figure 4: LLM SATD Classification Results — Notebook 06 and Notebook 11

Key Finding: RQ5 — ML-Specific vs Traditional Software Patterns

- **Average Severity by ML Category — first severity analysis in SATD literature:** NLP 2.86 (*highest*) · AutoML/RL 2.85 · Deep Learning 2.83 · MLOps 2.76 · Computer Vision 2.70 (*lowest*)
- **Overturing prior work:** Bhatia et al. found Requirement Debt most common (34%) — LLM semantic reclassification reveals this was a keyword artefact. Many TODO comments are code quality issues, correctly reclassified as Code Debt when context is considered.

SDLC Phase Analysis

SDLC Phase Analysis - 526 Repositories

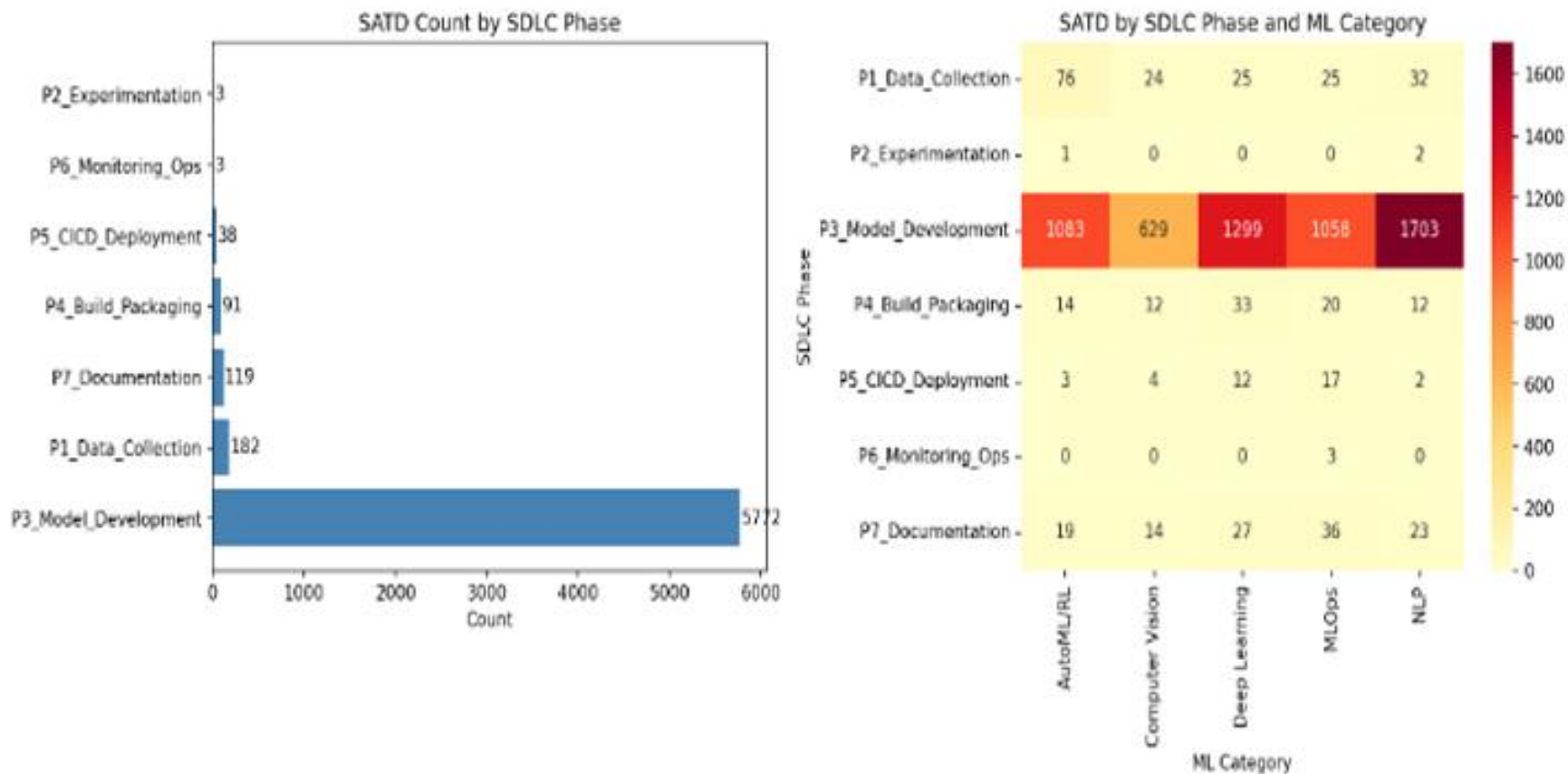


Figure 5: SATD Distribution by SDLC Phase and ML Category — 526 Repositories

SDLC Phase Analysis

Table 15: SATD by SDLC Phase — 526 Repositories

SDLC Phase	Total SATD	% of Total	Note
P3 — Model Development	5,772	93.0%	All from Python source
P1 — Data Collection	182	2.9%	AutoML/RL highest (76)
P7 — Documentation	119	1.9%	Non-Python only
P4 — Build & Packaging	91	1.5%	Deep Learning leads (33)
P5 — CI/CD & Deployment	38	0.6%	Non-Python only
P6 — Monitoring & Operations	3	0.05%	MLOps only

Critical finding: P7 and P5 debt — 157 instances total — are **entirely invisible** without multi-artifact scanning. MLOps holds the only P6 Monitoring debt, confirming its unique operational exposure

Limitations & Future Work

Table 16: Limitations and Future Research Directions

Limitation	Impact	Future Direction
L1: Corpus limited to 526 high-activity open-source repositories selected using four quality criteria	Findings may not generalize to private or enterprise ML systems	F1: Extend the study to private and industrial ML repositories for broader validation
L2: LLM classification applied to a stratified sample of 2,803 comments rather than all 6,027 Python records because of API rate limits	Severity distributions are based on a sample, not the complete dataset	F2: Apply the LLM classifier to the full dataset using a locally hosted model to bypass API constraints
L3: Python scanning limited to two directory levels and 50 files per repository	SATD counts represent conservative lower bounds	F3: Perform full-depth Python scanning with uniform coverage across all 526 repositories
L4: Non-Python scanning performed only at the repository root level, missing deeply nested CI/CD configurations	The observed 4.1% underestimation is itself a lower bound on the true non-Python debt gap	F4: Conduct full-repository multi-artifact scanning to more accurately quantify Python-only underestimation
L5: Eight behavioural clusters were labelled through researcher interpretation of TF-IDF terms	Cluster semantics may not fully capture developer intent	F5: Validate cluster labels through practitioner interviews with MLOps and NLP developers
L6: Novel debt types (Experiment Tracking, Token Budget, Monitoring) have low instance counts (1–7)	Limited statistical support for the new categories	F6: Track these novel debt types longitudinally to investigate delayed but persistent accumulation patterns

Conclusions

- **Contribution 1 — First statistically validated SATD comparison across ML sub-categories** $\chi^2=1204.66$ ($p \approx 0$) and $H=25.26$ ($p=0.000045$) confirm that ML sub-categories differ in both volume and composition of debt. MLOps is the structural outlier. Domain-specific debt signatures identified for all five sub-categories.
- **Contribution 2 — First LLM-based SATD classifier with severity grading** 0% false positive rate (vs 44% for prior NLP-based detector). Six novel ML-specific debt categories validated. First-ever per-instance severity analysis — revealing that severity and volume are independent dimensions.
- **Contribution 3 — First evidence of non-Python debt in ML repositories** Python-only analysis underestimates total SATD by 4.1% (up to 6.2% for Deep Learning). All five non-Python artifact types carry measurable debt invisible to all five prior reference studies.
- **Contribution 4 — First SDLC phase-level analysis in ML repositories** 93% of SATD accumulates in the Model Development phase. CI/CD and Documentation debt only visible via multi-artifact scanning. Two previously unnamed behavioural debt archetypes (Path Management Debt, Import Suppression Debt) identified through unsupervised clustering.

Acknowledgement & References

Acknowledgements

- **Supervisor:** Prof. Dr. sc. ing. Boriss Mišņevs — Transport and Telecommunication Institute
- **Groq API (LPU Inference)** — enabling high-throughput Llama 3.1 classification at scale
- **GitHub** — open-source repository data via public API

Replication Package: github.com/vidhumol/ml-technical-debt-analysis (*All 12 notebooks, raw data, processed results, figures*)

Acknowledgement & References

Key References:

- Sculley et al. (2015) — Hidden Technical Debt in Machine Learning Systems. *NeurIPS 2015*
- Bhatia et al. (2025) — An Empirical Study of SATD in Machine Learning Software. *ACM TOSEM*
- Selvanayagam et al. (2026) — SATD in LLM Software: Empirical Comparison. *arXiv:2601.06266*
- Pepe et al. (2024) — Taxonomy of SATD in Deep Learning Systems. *arXiv:2409.11826*
- Bavota & Russo (2016) — Large-Scale Empirical Study on SATD. *MSR 2016*
- Potdar & Shihab (2014) — Exploratory Study on Self-Admitted Technical Debt. *ICSME 2014*
- Cunningham (1992) — The Wycash Portfolio Management System. *ACM SIGSOFT AN*

THANK YOU FOR ATTENTION!

Vidhumol Pandippilly Antony | Master defence Presentation, 2026

Vidhumol04@gmail.com

Supervised by Dr.sc.ing. Boriss Misnevs

Answers to Reviewer's Questions

1. How can the research aim, object, subject, and tasks of the thesis be formulated?

While the reviewer notes that these formal elements were not explicitly defined in the thesis text, they can be formulated based on the provided methodology and research questions:

- **Research Aim:** To conduct a multi-modal empirical analysis of Self-Admitted Technical Debt (SATD) to understand its volume, composition, and severity across five distinct machine learning sub-categories.
- **Object of Research:** The phenomenon of technical debt (specifically SATD) within the lifecycle of open-source machine learning software development.
- **Subject of Research:** The comparative patterns, distribution, and semantic characteristics of SATD in 526 repositories belonging to Deep Learning, NLP, Computer Vision, MLOps, and AutoML/RL domains.
- **Research Tasks:**
 - Collect a diverse corpus of 526 ML and non-ML repositories.
 - Extract SATD instances from Python and non-Python artifacts using an expanded set of keywords.
 - Develop and apply an LLM-based semantic classifier to determine debt categories and severity.
 - Utilize unsupervised clustering to discover latent behavioral debt patterns.
 - Statistically validate differences in debt volume and composition across ML sub-categories.

Answers to Reviewer's Questions

2.How can the results be reproduced if the study depends on live GitHub API data?

- Although the study relies on the GitHub API, which tracks "live" and constantly changing data, the results can be reproduced through the following measures provided by the author:
- **Archived Datasets:** The author saved all extracted data into static CSV files (e.g., `repo_metadata.csv`, `satd_results.csv`, `static_analysis_results.csv`), which are stored in the project's GitHub home.
- **Replication Package:** A complete replication package including all 12 Jupyter notebooks and the processed data is available at a public repository (github.com/vidhumol/ml-technical-debt-analysis).
- **Fixed Snapshots:** During the baseline phase, repositories were cloned at a specific "current snapshot" (depth 1) to ensure the analysis was performed on a fixed version of the code at that point in time.

Answers to Reviewer's Questions

3. Why are some results presented as “High”, “Medium”, and “Low” instead of exact numerical values?

- The use of categorical labels instead of exact numerical values for some results is a result of the **Baseline ML Classifier** (Notebook 05). These labels represent a **composite debt severity score** calculated using a rule-based scoring function.
- **The Scoring Function:** It combines four weighted signals: SATD count, average cyclomatic complexity, average issue resolution days, and Flake8 violation counts.
- **The Thresholds:** A total score over 10 is classified as **High**, 7–9 as **Medium**, and below 7 as **Low**.
- **Purpose:** These labels were created to simplify multi-modal data into a target variable for training and evaluating predictive models, such as Random Forest and Logistic Regression.

4. What literature search strategy, databases, keywords, and selection criteria were used for selecting the sources?

- According to the **Reviewer's assessment**, the thesis failed to provide these specific details. The reviewer noted that while the literature review was logically structured and used relevant, recent sources, it was primarily **descriptive**.
- I completely agree with the reviewer. Thank you for the comment!